山东大学 计算机科学与技术 学院

云计算技术 课程实验报告

学号：202200130077	姓名：薛润泽	班级： 智能 22
实验题目：RSA 数字签名		
实验学时：2	实验日期： 2025.5.12	

实验目的：理解 RSA 数字签名算法的原理及其在信息安全中的应用；掌握 RSA 算法的密钥生成、签名生成与验证过程；通过编程实现 RSA 数字签名算法，并分析其性能与安全性。

具体内容：

1）基于 RSA 算法生成公钥和私钥对。

2）使用私钥对指定消息进行数字签名。

3）使用公钥验证数字签名的有效性。

4）分析不同密钥长度对算法性能的影响。

硬件环境：

计算机一台

软件环境：

Linux 或 Windows

实验步骤与内容：

1、基于 RSA 算法生成公钥和私钥对

（1）随机生成数，然后通过素数检测法-米勒-拉宾素性测试检测（费马小定理，二次探测定理）寻找两个大素数 p,q。Mod n 中的 n = p*q

（2）指定公钥 e = 65537，使用 p,q 计算欧拉函数，使用拓展欧几里得算法求私钥 d

2、使用私钥对指定消息进行数字签名

使用哈希函数，将目标文本映射为一个值 c，然后算 m = c^d mod n，结果就是数字签名后的消息

3、对 m 计算 r = m^e mod n，结果 r 是公钥验证数字签名后的结果。如果 r =c 说明签名有效，否则无效

4、指定不同密钥长度进行数字签名的加密解密，同时记录运行时间

结论分析与体会：

使用要求长度为 5, 10, 11, 12, 13, 14 的密钥进行数字签名结果如下

```
D:\大学\大三下\云计算技术\实验\实验9>python main.p
content: I love cloud computing technology.
bits: 5
模数 527
公钥 65537
私钥 353
hash num  403
真实哈希值为 403
数字签名解密后的哈希值为 403
数据有效
用时:  0.0100729465484619 14
bits: 10
模数 361567
公钥 65537
私钥 194633
hash num  64397
真实哈希值为 64397
数字签名解密后的哈希值为 64397
数据有效
用时:  0.1756882667541504
bits: 11
模数 3106399
公钥 65537
私钥 1682873
hash num  2563101
真实哈希值为 2563101
数字签名解密后的哈希值为 2563101
数据有效
用时:  7.9917488098144 53
bits: 12
模数 8562391
公钥 65537
私钥 8226433
hash num  3894910
真实哈希值为 3894910
数字签名解密后的哈希值为 3894910
数据有效
用时:  140.86505031585693
bits: 13
模数 57911519
公钥 65537
私钥 1022993
hash num  19249523
真实哈希值为 19249523
数字签名解密后的哈希值为 19249523
数据有效
用时:  6.216297626495361
```

```
bits: 13
模数 57911519
公钥 65537
私钥 1022993
hash num  19249523
真实哈希值为 19249523
数字签名解密后的哈希值为 19249523
数据有效
用时：  6.216297626495361
bits: 14
模数 108730003
公钥 65537
私钥 45008369
hash num  107957261
```

不同密钥长度对算法性能的影响：
5 位长度用时 0.01007s
10 位密钥长度用时 0.17568s
11 位密钥长度用时 7.99174s
12 位密钥长度用时 140.86505s
13 位密钥长度用时 6s
14 位密钥长度用时过久。

代码附录：

key_generator.py

```python
import random
import math

def is_prime_miller_rabin(n, k=40): # 隐藏接口
    if n <= 1:
        return False
    if n == 2 or n == 3:
        return True
    if n % 2 == 0:
        return False
    s = 0
    d = n - 1
    while d % 2 == 0:
        d //= 2
        s += 1
    for _ in range(k):
        a = random.randrange(2, n - 1)
```

```
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue


        for _ in range(s - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True


def generate_large_prime(bits):
    if bits < 2:
        raise ValueError("位数过小")
    while True:
        lower_bound = 1 << (bits - 1)
        upper_bound = (1 << bits) - 1

        candidate = random.randint(lower_bound, upper_bound)


        if candidate % 2 == 0:
            candidate += 1
            if candidate > upper_bound:
                continue
        num_miller_rabin_rounds = 40
        if bits > 512:
             num_miller_rabin_rounds = 64


        if is_prime_miller_rabin(candidate, k=num_miller_rabin_rounds):
            return candidate


def exgcd(a, b): # 隐藏接口
    if a == 0:
        return b, 0, 1
    else:
        x_prev, x_curr = 1, 0
        y_prev, y_curr = 0, 1
```

```
        while b != 0:
            quotient = a // b
            a, b = b, a % b
            x_prev, x_curr = x_curr, x_prev - quotient * x_curr
            y_prev, y_curr = y_curr, y_prev - quotient * y_curr
        return a, x_prev, y_prev


def calculate_private_key_d(e, phi_n):
    gcd, d_temp, y_prev = exgcd(e, phi_n)
    if gcd != 1:
        print("不互质")
        return None
    d = (d_temp % phi_n + phi_n) % phi_n
    return d
```

Hash_mapping.py

```
def hash_map(input_string: str) -> int:
    if not isinstance(input_string, str):
        raise TypeError("wrong_type")
    hash_value = 0
    prime_multiplier = 31
    modulus = 2**32
    for char in input_string:
        hash_value = (hash_value * prime_multiplier + ord(char)) % modulus
    return hash_value
```

Main.py

```
from key_generator import generate_large_prime, calculate_private_key_d
from hash_mapping import hash_map
import time
#generate_large_prime(bits)
#calculate_private_key_d(e, phi_n)
#hash_map(type=string)
if __name__ == '__main__':
```

```
e = 65537 # 公钥
bits_list =[5, 10, 11, 12, 13, 14]
content = "I love cloud computing technology. "
print("content:", content)
for bits in bits_list:
    start_time = time.time()
    p = generate_large_prime(bits)
    q = generate_large_prime(bits)
    while p== q:
        q = generate_large_prime(bits)
    phi_n = (p-1) *(q-1)
    mod_number = p * q
    d = calculate_private_key_d(e, phi_n)
    print('bits:', bits)
    print("模数", mod_number)
    print("公钥", e)
    print("私钥", d)
    hash_num = hash_map(content) % mod_number
    print("hash num ", hash_num)
    encoding = pow(hash_num, d) % mod_number
    decoding = pow(encoding, e) % mod_number
    print("真实哈希值为", hash_num)
    print("数字签名解密后的哈希值为",decoding)
    if(hash_num == decoding):
        print("数据有效")
    else:
        print("数据无效")
    end_time = time.time()
    elapsed_time = end_time - start_time
    print("用时：", elapsed_time)
```